S

I am working on a Pygame-based Tug of War game where the player must rapidly alternate between pressing the 'A' and 'D' keys to pull the rope to the left, while the computer opponent pulls to the right.

The deadlock in your current logic happens because `is_pull_locked` relies on a strict, idealized sequence of inputs: **Press A → Release A → Press D → Release D**.

When players mash keys rapidly, they almost always overlap their inputs (e.g., pressing 'D' *before* fully releasing 'A'). When this happens:

1. 'A' is pressed (`is_pull_locked` becomes `True`).
2. 'D' is pressed, but the input is completely ignored because `is_pull_locked` is still active.
3. 'A' is released, which frees the lock.
4. The player is now physically holding 'D' down and goes to press 'A' again. Since the system never registered 'D', the `last_key` is still 'A'. Their new 'A' press is ignored because `event.key == self.last_key` .

The player is now completely out of sync with the game's state machine, leading to the "deadlock" feeling.

The Fix: Edge-Triggered Alternation

To make mashing feel responsive and reliable, you should remove the `KEYUP` logic and the `is_pull_locked` boolean entirely. Instead, evaluate the input purely on the `KEYDOWN` edge. As long as the newly pressed key is one of the valid targets and is *different* from the `last_key` , it counts as a successful alternating pull.

Here is the refactored code:

Python

```
import pygame
```

```
class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
        self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)")
        self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTE

        # Track only the last pressed key to enforce alternating inputs
        self.last_key = None

        self.winner = None
        self.game_state = "PLAYING"
        # ...

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        # Purely edge-triggered logic (KEYDOWN only)
        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_a, pygame.K_d):
                # If the key is the opposite of the last successful press, pull
                if event.key != self.last_key:
                    self.rope.pull_left(1.0)
                    self.last_key = event.key
```

By removing the lock state, overlapping keys won't penalize the player. If they accidentally press 'A' twice in a row, the second press is simply ignored, but they can immediately recover by pressing 'D', keeping the rhythm intact without freezing.

Want to add a stamina mechanic?

↪ Yes

Now let's work on Task 2: Implement Dynamic AI Panic Surges.

Currently in `game/game_engine.py`, the comput

To implement the AI "panic surge," we need to calculate how close the rope's current position is to the player's winning threshold. By mapping this distance to a `panic_factor` (from `0.0` for safe to `1.0` for imminent defeat), we can dynamically scale down the computer's cooldown and scale up its pull strength.

Python

```python
import pygame
import random

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
        self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)")
        self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTE

        # Input tracking
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

        # AI Base configuration
        self.base_ai_cooldown = 180
        self.min_ai_cooldown = 70  # Fastest the AI can pull when in max panic
        self.base_ai_strength = 1.0
        self.max_ai_strength = 2.0  # Hardest the AI can pull when in max panic

        # Runtime AI state
        self.current_ai_cooldown = self.base_ai_cooldown
        self.last_computer_pull = pygame.time.get_ticks()

        # Assuming the rope starts at width // 2
        self.center_start_x = width // 2

    def reset(self):
        self.rope = Rope(self.width, self.height)
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"
        self.current_ai_cooldown = self.base_ai_cooldown
        self.last_computer_pull = pygame.time.get_ticks()
```

```
    def update(self):
        if self.game_state != "PLAYING":
            return

        now = pygame.time.get_ticks()

        # Calculate how close the player is to winning.
        # Assuming self.rope.x tracks the center of the rope.
        total_distance_to_lose = self.center_start_x - self.rope.left_win_x
        current_distance_to_lose = self.rope.x - self.rope.left_win_x

        # Panic factor: 0.0 (middle) to 1.0 (at the left_win_x edge)
        # Clamped between 0 and 1 so the AI doesn't panic if it is winning.
        panic_factor = 1.0 - (current_distance_to_lose / total_distance_to_lose
        panic_factor = max(0.0, min(1.0, panic_factor))

        # Dynamically scale cooldown and strength based on panic factor
        cooldown_range = self.base_ai_cooldown - self.min_ai_cooldown
        self.current_ai_cooldown = self.base_ai_cooldown - (cooldown_range * pa

        if now - self.last_computer_pull >= self.current_ai_cooldown:
            # Strength increases as panic increases
            strength_range = self.max_ai_strength - self.base_ai_strength
            current_strength = self.base_ai_strength + (strength_range * panic_

            # Apply standard variance to the new calculated strength
            computer_variance = random.uniform(current_strength - 0.3, current_
            self.rope.pull_right(computer_variance)
            self.last_computer_pull = now

        result = self.rope.check_winner()
        if result:
            self.winner = result
            self.game_state = "GAME_OVER"
```

Want to add visual feedback when the AI panics?

↪ Yes

Now let's work on Task 3: Implement Rope Tension & Puller Leaning Animations.

Currently, the rope and puller characters are visua...

To implement the dynamic visual feedback, we need to add rotation capabilities to the `Puller` characters using Pygame's surface transformation, and break the `Rope` drawing into segments to allow for mathematical sag and random jitter.

We will track `tension` and `momentum` in the `GameEngine` and pass them to the respective `render` methods.

1. Update `game/player.py`

We will render the puller onto a transparent temporary surface, rotate it based on the current momentum, and then blit it back.

Python

```python
import pygame

class Puller:
    """Represents a puller character anchor on either side of the rope."""

    def __init__(self, x, y, color, label, face_dir):
        self.x = x
        self.y = y
        self.color = color
        self.label = label
        self.font = pygame.font.SysFont(None, 24)
        # face_dir: -1 for Player (left side), 1 for Computer (right side)
        self.face_dir = face_dir

    def render(self, surface, lean_intensity=0.0):
        """Draw avatar and label with dynamic leaning."""
        # Create a transparent surface for the character body to allow rotatior
        temp_surf = pygame.Surface((80, 120), pygame.SRCALPHA)

        # Body (drawn centered on the temp surface)
        body_rect = pygame.Rect(20, 40, 40, 70)
        pygame.draw.rect(temp_surf, self.color, body_rect, border_radius=6)

        # Head
        pygame.draw.circle(temp_surf, (240, 210, 180), (40, 25), 16)

        # Calculate lean angle based on intensity (max 25 degrees)
        # face_dir ensures they lean backward relative to their side
```

```
        angle = 25 * lean_intensity * -self.face_dir

        # Rotate and keep centered
        rotated_surf = pygame.transform.rotate(temp_surf, angle)
        new_rect = rotated_surf.get_rect(center=(self.x, self.y - 15))
        surface.blit(rotated_surf, new_rect)

        # Name / control tag
        label_surf = self.font.render(self.label, True, (240, 240, 240))
        surface.blit(label_surf, (self.x - label_surf.get_width() // 2, self.y
```

2. Update `game/rope.py`

We will replace the single line draw with a segmented line that can calculate a parabolic sag for low tension, and apply random Y-axis jitter for high tension.

Python

```python
import pygame
import random

class Rope:
    def __init__(self, width, height):
        self.screen_width = width
        self.center_y = height // 2
        self.x = width // 2
        self.left_win_x = width // 2 - 200
        self.right_win_x = width // 2 + 200

    def pull_left(self, amount):
        self.x -= amount

    def pull_right(self, amount):
        self.x += amount

    def check_winner(self):
        if self.x <= self.left_win_x: return "PLAYER"
        if self.x >= self.right_win_x: return "COMPUTER"
        return None

    def render(self, surface, tension=0.0):
        """Render rope with dynamic sag and vibration."""
        start_pos = (60, self.center_y)
        end_pos = (self.screen_width - 60, self.center_y)
```

```
        points = []
        num_segments = 20

        for i in range(num_segments + 1):
            t = i / num_segments
            seg_x = start_pos[0] + (end_pos[0] - start_pos[0]) * t
            seg_y = self.center_y

            # Apply Sag (prominent when tension is low)
            sag_amount = (1.0 - tension) * 40  # Max 40px sag
            # Parabolic curve formula for the sag
            sag_offset = sag_amount * (4 * t * (1 - t))
            seg_y += sag_offset

            # Apply Vibration (prominent when tension is high)
            if tension > 0.6:
                vibe_intensity = (tension - 0.6) * 2.5
                seg_y += random.uniform(-4, 4) * vibe_intensity

            points.append((seg_x, seg_y))

        pygame.draw.lines(surface, (180, 140, 90), False, points, 10)

        # Center marker flag
        flag_x = points[num_segments // 2][0]
        flag_y = points[num_segments // 2][1]

        # Center marker line
        pygame.draw.line(surface, (200, 30, 30), (self.x, flag_y - 20), (self.x
```

3. Hooks in `GameEngine`

You need to initialize the tension/momentum tracking, bump them when a pull occurs, decay them naturally in `update()`, and pass them to the rendering methods.

Python

```
# In __init__ and reset(), update Puller initializations with face_dir and
self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)", face_
self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTER",

self.tension = 0.0
self.player_momentum = 0.0
```

```
self.computer_momentum = 0.0

# In handle_event() under the successful player pull:
self.rope.pull_left(1.0)
self.player_momentum = min(1.0, self.player_momentum + 0.3)
self.tension = min(1.0, self.tension + 0.15)

# In update() under the computer's successful pull:
self.rope.pull_right(computer_variance)
self.computer_momentum = min(1.0, self.computer_momentum + 0.3)
self.tension = min(1.0, self.tension + 0.15)

# Also in update(), add the decay logic so tension and leaning settle down
if self.game_state == "PLAYING":
    self.tension = max(0.0, self.tension - 0.015)
    self.player_momentum = max(0.0, self.player_momentum - 0.02)
    self.computer_momentum = max(0.0, self.computer_momentum - 0.02)

# Finally, in your main render loop, pass the variables:
self.rope.render(surface, tension=self.tension)
self.player.render(surface, lean_intensity=self.player_momentum)
self.computer.render(surface, lean_intensity=self.computer_momentum)
```

Add a sound system?

↪ Yes

Now let's work on Task 4: Implement a Match Timer and Sudden Death Mode.

Requirement:

To implement the Match Timer and Sudden Death mode, we need to integrate time tracking into the engine. When the elapsed time crosses the 45-second threshold, we activate the sudden death state, which applies a `2.0x` multiplier to both the player's pulling action in `handle_event` and the computer's pulling action in `update()`.

Here are the updated core methods for `GameEngine`:

Python

```python
import pygame
import random

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height
        self.rope = Rope(width, height)
        self.player = Puller(90, height // 2, (50, 120, 220), "PLAYER (A/D)", 1
        self.computer = Puller(width - 90, height // 2, (220, 80, 50), "COMPUTE

        # Game State
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

        # Timer & Sudden Death
        self.match_start_time = pygame.time.get_ticks()
        self.is_sudden_death = False
        self.sudden_death_threshold = 45000  # 45 seconds in milliseconds

        # Fonts for UI
        self.font_timer = pygame.font.SysFont(None, 36)
        self.font_banner = pygame.font.SysFont(None, 48)

        # AI & Physics tracking (from previous tasks)
        self.base_ai_cooldown = 180
        self.min_ai_cooldown = 70
        self.base_ai_strength = 1.0
        self.max_ai_strength = 2.0
        self.current_ai_cooldown = self.base_ai_cooldown
        self.last_computer_pull = pygame.time.get_ticks()
        self.center_start_x = width // 2

        self.tension = 0.0
        self.player_momentum = 0.0
        self.computer_momentum = 0.0

    def reset(self):
        self.rope = Rope(self.width, self.height)
        self.last_key = None
        self.winner = None
        self.game_state = "PLAYING"

        # Reset Timer and AI state
        self.match_start_time = pygame.time.get_ticks()
```

```
        self.is_sudden_death = False
        self.current_ai_cooldown = self.base_ai_cooldown
        self.last_computer_pull = pygame.time.get_ticks()

        self.tension = 0.0
        self.player_momentum = 0.0
        self.computer_momentum = 0.0

    def handle_event(self, event):
        if self.game_state != "PLAYING":
            if event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                self.reset()
            return

        if event.type == pygame.KEYDOWN:
            if event.key in (pygame.K_a, pygame.K_d):
                if event.key != self.last_key:
                    # Apply sudden death multiplier to player
                    pull_power = 2.0 if self.is_sudden_death else 1.0
                    self.rope.pull_left(pull_power)
                    self.last_key = event.key

                    # Visual feedback hooks
                    self.player_momentum = min(1.0, self.player_momentum + 0.3)
                    self.tension = min(1.0, self.tension + 0.15)

    def update(self):
        if self.game_state != "PLAYING":
            return

        now = pygame.time.get_ticks()
        elapsed = now - self.match_start_time

        # Check for Sudden Death
        if elapsed >= self.sudden_death_threshold and not self.is_sudden_death:
            self.is_sudden_death = True

        # AI Panic logic
        total_distance_to_lose = self.center_start_x - self.rope.left_win_x
        current_distance_to_lose = self.rope.x - self.rope.left_win_x
        panic_factor = 1.0 - (current_distance_to_lose / total_distance_to_lose
        panic_factor = max(0.0, min(1.0, panic_factor))

        cooldown_range = self.base_ai_cooldown - self.min_ai_cooldown
        self.current_ai_cooldown = self.base_ai_cooldown - (cooldown_range * pa

        # Computer Pull execution
        if now - self.last_computer_pull >= self.current_ai_cooldown:
            strength_range = self.max_ai_strength - self.base_ai_strength
```

```
            current_strength = self.base_ai_strength + (strength_range * panic_

            # Apply sudden death multiplier to computer
            if self.is_sudden_death:
                current_strength *= 2.0

            computer_variance = random.uniform(current_strength - 0.3, current_
            self.rope.pull_right(computer_variance)
            self.last_computer_pull = now

            # Visual feedback hooks
            self.computer_momentum = min(1.0, self.computer_momentum + 0.3)
            self.tension = min(1.0, self.tension + 0.15)

        # Decay visual feedback
        self.tension = max(0.0, self.tension - 0.015)
        self.player_momentum = max(0.0, self.player_momentum - 0.02)
        self.computer_momentum = max(0.0, self.computer_momentum - 0.02)

        # Win Condition
        result = self.rope.check_winner()
        if result:
            self.winner = result
            self.game_state = "GAME_OVER"

    def render(self, surface):
        surface.fill((30, 30, 40)) # Background

        # Render Game Entities
        self.rope.render(surface, tension=self.tension)
        self.player.render(surface, lean_intensity=self.player_momentum)
        self.computer.render(surface, lean_intensity=self.computer_momentum)

        # Render UI
        if self.game_state == "PLAYING":
            now = pygame.time.get_ticks()
            elapsed_sec = (now - self.match_start_time) // 1000

            # Timer display
            timer_text = f"Time: {elapsed_sec}s"
            timer_surf = self.font_timer.render(timer_text, True, (255, 255, 25
            surface.blit(timer_surf, (self.width // 2 - timer_surf.get_width()

            # Sudden Death Banner
            if self.is_sudden_death:
                # Flashing effect (blinks every 250ms)
                if (now // 250) % 2 == 0:
                    banner_surf = self.font_banner.render("SUDDEN DEATH! (2X Pl
                    surface.blit(banner_surf, (self.width // 2 - banner_surf.ge
```

```
        elif self.game_state == "GAME_OVER":
            over_surf = self.font_banner.render(f"{self.winner} WINS!", True, (
            surface.blit(over_surf, (self.width // 2 - over_surf.get_width() //
            restart_surf = self.font_timer.render("Press 'R' to Rematch", True,
            surface.blit(restart_surf, (self.width // 2 - restart_surf.get_widt
```

Add a background color shift?